One Cache, Any Catalog: Matryoshka Gist Pointers for Calibrated, Order-Invariant Tool Selection at Scale

Anonymous Author(s)

Abstract

Tool-using agents must pick one action from catalogs of hundreds to thousands of tools, and must recognise when no tool fits. Listwise LLM rerankers and pointer-style routers make the finest distinctions, but they re-read every tool description on every request, they cannot fit large catalogs in context, and their decisions change when the same tools are listed in a different order. We present *Matryoshka Gist Pointers* (MGP), a decision model that reads a catalog once. Each tool is encoded in isolation, with catalog-independent positions, and summarised into a small set of nested *gist* tokens whose key–value states are cached per tool. A request attends to the cached gists of any subset of tools, in any order, and a pointer head scores every tool plus an explicit abstain option; split-conformal prediction turns the scores into execute / ask / abstain decisions with a coverage guarantee. We prove, and verify numerically, that MGP's scores are exactly equivariant to catalog order (max deviation 1.6×10^{-6} in fp32, versus 0.246 for a pointer router), that per-tool caches compose exactly, and that one cache serves every gist budget, which yields a two-level coarse-to-fine decision from a single model. On ToolRet, a first-stage retriever recovers the gold tool in its top-16 for only 47.7% of queries but in its top-1,024 for 91.0%; MGP reads 1,024 tools in 110 ms on a laptop GPU, where a pointer router cannot run beyond 64. Because the request is the only part that changes, the same text cache also serves spoken requests and requests about a screenshot: on an omni backbone (Gemma 4 E2B) speech and image soft tokens enter the request block and every property above still holds. [Accuracy at 16/64/256/1024 tools vs. rerankers, abstention AUROC and conformal coverage on BFCL / When2Call; speech vs. transcript gap; screenshot pointing.]

Keywords

tool retrieval, listwise reranking, KV caching, context compression, conformal prediction, LLM agents, spoken and multimodal requests

ACM Reference Format:

Anonymous Author(s). 2026. One Cache, Any Catalog: Matryoshka Gist Pointers for Calibrated, Order-Invariant Tool Selection at Scale. In *Proceedings of Proceedings of the 50th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '27).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

An agent that can call thousands of tools faces a ranking problem with three unusual requirements. First, distinctions are fine-grained: `create_calendar_event` and `create_reminder` share most of their vocabulary, and the right choice depends on argument structure and intent. Second, the catalog is large and changes per user and per session (MCP servers are attached and detached at run time), so a model that memorises tool identities in its vocabulary [42] must be retrained whenever the catalog changes. Third, the decision must be *calibrated*: when no tool fits, the agent should ask or abstain rather than act [21, 27].

Existing systems give up one of these. Bi-encoders [32, 40] scale but blur near-duplicates; ToolRet reports that strong IR embedders reach only 34–47 nDCG@10 on tool retrieval [32]. Cross-encoders and listwise LLM rerankers [25, 34, 41] are accurate but cost one long prefill per request over every candidate, and listwise models are sensitive to the order of the candidates [17, 50]. Pointer-style routers that read the request and the full menu and point at one option (the "Kev" family of decision models) add an abstain option but inherit both problems: on our data 85% of every prefill is static menu text, reversing the menu shifts the chosen probability by up to 0.246, and the menu cannot exceed a few dozen tools.

Concurrent systems work cache tool schemas per tool [3, 7] or block cross-candidate attention in rerankers [4, 31]. They show that isolation costs little accuracy, but they generate free-form calls, keep the full schema in cache, and do not offer calibrated abstention or an exact order guarantee. MGP is a *decision* model built around four ideas:

(1) **Block-isolated tools at catalog-independent positions**, with the request at a fixed position offset, so tool states never depend on which other tools are present or where they are listed (§3.2).
(2) **Nested (Matryoshka) gists**: g learned summary tokens per tool, trained with random budgets so the first b gists suffice on their own. A tool's cache is g KV entries (0.047 MB in bf16 for g=4 on a 0.5B backbone) (§3.3).
(3) **Two-level decisions from one cache**: a coarse pass over gist 1 of every tool and a fine pass over all gists of the top-M, i.e. retrieval and listwise reranking in one model (§3.5).
(4) **A pointer head with abstain and split-conformal sets**, giving execute / ask / abstain actions whose coverage does not depend on how the client orders its tools (§3.7).
(5) **Any request modality over one text cache**: on an omni backbone the request may be speech or carry a screenshot; its soft tokens sit in the request block, so the catalog cache, the proofs and the conformal guarantee are unchanged (§3.6).

We contribute (i) the architecture and its training recipe; (ii) three properties—composition, order equivariance, budget nesting—with proofs and numerical checks (§4); (iii) an evaluation on ToolRet at 16 to 1,024 candidates, on abstention benchmarks, on spoken and screenshot requests, and on latency and memory (§5); and (iv) code, weights and per-tool caches.

SIGIR '27, San Jose, CA, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Related Work

Tool retrieval and selection. Tool-learning benchmarks such as ToolBench [23], Gorilla/BFCL [21, 22] and When2Call [27] evaluate whether a model calls the right tool and whether it declines when none fits. ToolRet [32] unifies 35 task sets over a 43k-tool corpus and shows that IR-strong embedders transfer poorly to tools; follow-ups expand tool documents and fine-tune embedders and rerankers (Tool-DE, Tool-Embed/Tool-Rank [40]). Training-free candidate compression (Toollery [35]) and learned tool embeddings (NTILC [13]) reduce what the LLM sees to a top-K list; a 2026 study [37] shows that end-to-end selection degrades even when top-K recall is near perfect. ToolGen [42] turns each tool into a vocabulary token, which removes the menu from the prompt but ties the model to a fixed catalog. MGP sits between these: it reads up to thousands of candidates listwise like a reranker, while the per-tool cost is paid once, offline.

Listwise reranking and order sensitivity. RankGPT [34], FIRST [25] and jina-reranker-v3 [41] rerank by reading candidates jointly; PE-Rank [18] compresses each passage to a single embedding token. LLMs are sensitive to candidate and option order [17, 50], and recent work exploits this as an attack surface on listwise recommenders [39]. Set-Encoder [31] achieves permutation invariance in a cross-encoder through inter-passage attention over `[INT]` tokens, and InvariRank [4] blocks cross-candidate attention with shared RoPE positions for LLM recommendation reranking. MGP uses the same isolation principle but adds per-tool caching across queries, nested compression, abstention and a conformal guarantee; InvariRank and Set-Encoder re-encode every candidate for every query.

Reusable and position-independent KV caches. Prefix caching only reuses an identical prefix. Parallel/position-independent encoding reuses chunks at arbitrary positions: Block-Attention [33] fine-tunes a model to read independently encoded blocks; APE [45] aligns parallel and sequential attention without training; EPIC [11], CacheBlend [46] and KVLink [44] recompute or link a few tokens to repair cross-chunk attention. For agents, ContextCache [30] and Nexus [3] transplant compiled schema caches (Nexus reports RoPE drift away from an anchor position), and CacheRouter [47] keeps the main model's prefix stable by routing through a side model. Closest to us, ReCache [7] caches each tool or skill schema with resource-wise attention and resource-local positions, and prunes visibility by layer and head; it matches dense attention in invocation F1 (82.3 vs. 82.4) with a 3.7× TTFT speed-up. ReCache keeps full (pruned) schemas, generates free-form invocations, and reports that a gist-compression baseline loses heavily (39.2 Inv-F1). MGP differs in four ways: (a) a fixed request offset gives *exact* rather than empirical order invariance; (b) per-tool caches are compressed to g nested gists, and we close the gist gap with self-distillation from the uncompressed path and a fine pass over the top-M; (c) one cache supports a two-level decision; (d) the output is a calibrated pointer distribution with abstention.

Context compression. Gist tokens [20], AutoCompressor [5], ICAE [8], Activation Beacon [48] and 500xCompressor [15] compress long contexts into a few soft tokens or KV entries. A systematic study finds gist compression near-lossless for many tasks but weak at exact recall [6], and GistPool revisits the design for long contexts [36]. Matryoshka representation learning [14] trains nested prefixes of an embedding to be usable at any size; we transfer the idea to gist sequences, where the nesting comes for free from causal attention within a block. Late-interaction retrieval (ColBERT [12]) keeps one vector per token; our late-interaction variant is an ablation.

Spoken and visual requests. Voice assistants classically route speech through ASR and a text intent classifier; end-to-end spoken language understanding removes the transcript, and Audio2Tool [28] evaluates tool calls from speech. GUI agents ground instructions to screen elements; OS-Atlas [43] provides cross-platform grounding data. Omni models such as Gemma 4 [9] and Qwen3-Omni [24] map audio and images into the language model's token space. We use that space directly: the request's soft tokens attend to a text tool cache that was built without them.

Uncertainty, abstention and conformal prediction. Split conformal prediction gives finite-sample coverage under exchangeability [2]; least-ambiguous sets (LAC) [29] threshold the softmax score. KnowNo [26] uses conformal sets over an LLM's options to decide when a robot should ask for help; conformal abstention bounds hallucination [1]; recent work applies conformal risk control to tool calls by role [38]. Distance-aware heads (SNGP [16]), deep ensembles and energy scores [19] are standard OOD signals and appear as head ablations. Because conformal coverage is a statement about scores, it is only meaningful for tool selection if the scores do not depend on presentation order; MGP makes that true by construction.

3 Method

3.1 Problem

A decision instance is a request x (text, and in later work audio or an image), a catalog $C = \{t_1, \ldots, t_N\}$ of tool descriptions, and the abstain option $\varnothing$. The model outputs $p(\cdot \mid x, C)$ over $C \cup \{\varnothing\}$ and an action $a \in \{\textsc{execute}(t), \textsc{ask}(S), \textsc{abstain}\}$.

3.2 Block-isolated tools

The input is a shared prefix P ("Options:"), then one block per tool, then the request. Token u in block k may attend to P and to earlier tokens of block k only; positions inside block k start at $|P|$ regardless of k (catalog-independent positions). Request tokens take positions $R, R{+}1, \ldots$ for a fixed offset R (we use R=640, larger than any tool block), and attend to P, to earlier request tokens, and to the *visible* tokens of every block (the gists, §3.3). The abstain option is a block like any other, so the model learns its representation.

3.3 Nested gists

After each tool's text we append g gist tokens $\gamma_1, \ldots, \gamma_g$ with learned embeddings. Gists attend to their own tool text and to earlier gists of the same tool (causal order), so the KV state of γ_j does not depend on $\gamma_{j'}$ for $j' > j$. The request sees only gists; a tool's cache is therefore its g gist KV entries per layer. During training we sample a budget $b \in \mathcal{B} = \{1, 2, 4, 8\}$ per step and let the request see only $\gamma_{1..b}$ of every tool, so each prefix is a usable summary.

Gist self-distillation. In the same step we run the request against the *full* tool tokens (no compression) and add $\mathrm{KL}(p_{\text{full}} \,\|\, p_{\text{gist}})$ with the full path detached, so compression is trained to preserve the uncompressed decision.

3.4 Pointer head and training objective

The request's last hidden state h_x and each tool's pooled visible state h_k (mean or attention pool over its gists) are layer-normalised and projected; the score is $s_k = \langle W_q h_x, W_o h_k \rangle / \sqrt{d}$, and $p = \mathrm{softmax}(s)$ over $C \cup \{\varnothing\}$. The loss is listwise cross-entropy plus a Brier term, the distillation term above, and (when used) teacher soft labels from a larger LLM scored over option letters with rotation averaging. Ablations replace the dot head with a Gaussian head (with KL), an RBF head with spectral normalisation, a 3-member ensemble scored by energy, or late interaction (MaxSim over per-token states).

3.5 Two-level decisions from one cache

Coarse: the request attends to γ_1 of all N tools (N keys per layer) and we keep the top-M by score. Fine: the request attends to all g gists of those M tools and the abstain block. Both passes read the same cache; only γ_1 must be resident for every tool, which supports memory tiering at 10k+ tools.

3.6 Spoken and screenshot requests

On an omni backbone the request block may begin with media: the backbone's own frozen audio or vision tower and projector map a waveform or an image to n soft tokens, which are placed after a short "Input:" prefix inside the request block, at positions $R, R{+}1, \ldots$. Tool blocks are untouched, so a catalog cache built from text serves text, speech and image requests alike. Backbones with per-layer token embeddings (Gemma 4 E2B/E4B look each token up a second time for every layer) receive these per-layer inputs computed from clean token ids, with gist, slot and media positions mapped to padding, the same rule the backbone uses for media positions. For UI pointing the options are screen regions described only by position and size, never by their text, so the screenshot must decide.

3.7 Conformal decisions

On a held-out calibration split we compute the LAC threshold $\hat{q}$ as the $\lceil (n{+}1)(1{-}\alpha) \rceil / n$ quantile of $1 - p(y_i)$. The set is $S(x) = \{c : p(c \mid x, C) \geq 1 - \hat{q}\}$; we EXECUTE if $S = \{t\}$, ABSTAIN if $S = \{\varnothing\}$ or empty, and ASK otherwise (the agent shows S to the user). We also report a normalised variant that conditions $\hat{q}$ on catalog size.

4 Properties

Let K_k, V_k denote the gist keys and values of tool k at a layer.

PROPOSITION 1 (COMPOSITION). *The KV states of block k are a function of (P, t_k) only. Hence caches computed for each tool separately and concatenated equal the caches of a joint pass over the catalog.*

SKETCH. By the mask, every token of block k attends only to P and to block k, and its positions are independent of k and of N. Induction over layers. □

PROPOSITION 2 (ORDER EQUIVARIANCE). *For any permutation π of the catalog, $s_{\pi(k)}(x, \pi C) = s_k(x, C)$ and $s_\varnothing$ is unchanged.*

SKETCH. Request tokens have fixed positions $R, \ldots$ and attend to the union of all visible gist keys, whose RoPE phases do not depend on catalog order. Softmax attention is a sum over keys, hence invariant to their order, so h_x is invariant; h_k depends on block k only (Prop. 1). The head is applied per option. □

COROLLARY 1 (ORDER-INVARIANT CONFORMAL SETS). *$S(x)$, as a set of tool identities, is invariant to catalog order; the split-conformal coverage $\Pr[y \in S(x)] \geq 1 - \alpha$ holds whatever order clients use, at calibration and at test time.*

PROPOSITION 3 (BUDGET NESTING). *The cache for budget b is the first b gist entries of the cache for budget g; two-level scoring with $M{=}N$ and $b{=}g$ equals single-pass scoring.*

Media requests. Propositions 1–3 and the corollary make no assumption about the content of the request block, so they hold verbatim when it contains speech or image soft tokens.

Numerical check (fp32, Qwen2.5-0.5B, untrained heads). Reversing a 16-tool catalog changes probabilities by at most 1.6×10^{-6} (block and gist layouts) versus 0.246 for the query-first pointer layout. Composed per-tool caches match a joint pass to $1.2{\times}10^{-6}$ (gists) and 1.8×10^{-7} (late interaction). Two-level scoring at full budget equals single-pass scoring exactly (0.0). Deviations are float summation order. With a speech request on a randomly initialised Gemma 4 with the real E2B layout (per-layer embeddings, sliding and global layers, audio and vision towers), order equivariance, composition and two-level nesting are exact in fp64 (0.0) and at 10^{-6} relative in fp32. [Same check on pretrained Gemma 4 E2B, 4-bit, T4.]

Complexity. A pointer router prefills $L_q + NL_t$ tokens per request: $O((L_q + NL_t)d^2 + (L_q + NL_t)^2 d)$. MGP prefills L_q tokens and attends to Nb cached keys: $O(L_q d^2 + L_q N b\, d)$, plus $O(L_q M g\, d)$ for the fine pass. On our data $L_q \approx 38$ and a 6-tool menu is ≈ 246 tokens.

5 Experimental Setup

Data. (i) ToolRet [32]: queries with gold tools; candidates are the top-K of a MiniLM first stage for $K \in \{16, 64, 256, 1024\}$, with the gold tool inserted only when computing oracle-menu accuracy (reported separately). The `toolace` subset is excluded because it overlaps our training source. (ii) Soro packages: 6-tool menus with look-alike near-misses and abstain cases, split by package. (iii) Abstention: BFCL irrelevance [21] and When2Call [27]. (iv) Spoken requests: held-out requests rendered with Kokoro TTS [10] across voices with noise and speed augmentation; each is paired with its transcript, so the text-to-speech gap is measured on identical decisions, and text twins of spoken test requests are removed from training. (v) Screenshot pointing: OS-Atlas UI rows with region-only options and absent-element abstain cases. Audio2Tool is used for evaluation only (non-commercial licence). Training data is MinHash-deduplicated against all test sets and audited for leakage.

Backbones. Qwen3-0.6B-Base for all text results; Gemma 4 E2B (4-bit, LoRA, frozen media towers) for the scaling run and all speech and screenshot results. Hybrid linear-attention backbones

Table 1: Recall of a MiniLM first stage on ToolRet (400 queries). A reader limited to K candidates cannot exceed this.

K	1	16	64	256	1024
Recall@K (%)	17.0	47.7	67.5	83.3	91.0

Table 2: Latency per request (ms, bf16, RTX 4050, 0.5B backbone, untrained heads). "–": out of memory (14.8k / 62k-token prompts).

Tools	6	16	64	256	1024
Pointer router (query first)	72	252	2204	–	–
Catalog first, cached text	56	65	86	–	–
Gist ×4, composed caches			102	86	204
MGP (nested ×8)			87	76	110
Two-level (1×N → 8×16)			524	164	361

(e.g. Gated DeltaNet layers) cannot express the block mask and are excluded.

Baselines. BM25; MiniLM; Qwen3-Embedding-0.6B [49]; Qwen3-Reranker-0.6B and jina-reranker-v3 [41] on the same candidates; a query-first pointer router with the same backbone and training data; Set-Encoder [31]; PE-Rank [18]; ReCache-style uncompressed per-tool caches (our block layout with full tool tokens visible, configuration "F"); ToolGen and Tool-Rank as reported numbers where protocols match.

Metrics. Top-1 accuracy and nDCG@10; abstain AUROC and F1; conformal coverage, set size and execute rate at $\alpha \in \{0.05, 0.1\}$; error-detection AUROC; latency per request and cache MB per tool. Three seeds; query-clustered bootstrap 95% CIs.

Hardware. Training on 2× NVIDIA T4 (fp16 autocast, fp32 master weights). Latency on an RTX 4050 laptop GPU (bf16), a T4, and CPU.

6 Results

6.1 Why read large catalogs

Table 1: a reranker limited to 16 candidates caps end-to-end accuracy at 47.7%; reading 1,024 lifts the ceiling to 91.0%.

6.2 Efficiency

Table 2. A tool's cache is 0.047 MB (g=4, bf16; 48 MB for 1,024 tools); late interaction needs 0.007 MB. At ≤1k tools a single pass is faster than two-level; two-level is for memory tiering at larger scale. [TODO: Repeat on T4 and CPU with 50 repetitions; add cache build time per tool.]

6.3 Accuracy at scale

6.4 Calibrated abstention

[Abstain AUROC/F1 on BFCL irrelevance and When2Call; coverage, set size and execute rate at $\alpha = 0.1$; order-shuffled coverage for the pointer router vs. MGP (the corollary predicts no change for MGP).]

Table 3: ToolRet top-1 accuracy (%) with MiniLM top-K candidates. [all cells]

K	16	64	256	1024
MiniLM / Qwen3-Emb	[]			
+ Qwen3-Reranker / jina-v3	[]			
Pointer router	[]	[]	–	–
Full-text block (F)	[]	[]	–	–
MGP single pass	[]			
MGP two-level	[]			

Table 4: Gemma 4 E2B, one text tool cache, three request modalities. [all cells]

	Transcript	Speech	Screenshot (UI)
Pointer router (A)	[]	[]	[]
MGP (KM)	[]	[]	[]

6.5 Spoken and screenshot requests

[Table 4: top-1 accuracy, abstain F1 on absent-element rows, conformal coverage per modality; the speech minus transcript gap is the cost of skipping ASR.]

6.6 Ablations

[Gists $g \in \{1, 2, 4, 8\}$; nested vs. fixed budget; distillation on/off; head type (dot, Gaussian, RBF-SN, ensemble-energy, late); attention pooling; accuracy by budget from one nested cache.]

7 Limitations

Isolation removes cross-tool comparison during encoding; tools are compared only through the request. Gist compression may blur exact identifiers (a known weakness of gist tokens [6], and the main risk identified by ReCache's gist baseline [7]); the fine pass and distillation are our mitigations, and Table 3 measures the residual gap. The block mask needs full softmax attention in every layer, which excludes hybrid linear-attention backbones and requires checking sliding-window models. Conformal coverage is marginal and assumes exchangeability between calibration and deployment requests; we calibrate per modality because speech and text score distributions differ. Spoken requests are synthetic (TTS); real recordings are evaluated only through Audio2Tool.

8 Conclusion

MGP turns a catalog into a set of small, composable, order-free caches and reads them as a calibrated pointer. [TODO: one-sentence headline result once Table 3 is filled.]

References

[1] Yasin Abbasi-Yadkori et al. 2024. Mitigating LLM Hallucinations via Conformal Abstention. *arXiv preprint arXiv:2405.01563* (2024).

[2] Anastasios N. Angelopoulos and Stephen Bates. 2023. Conformal Prediction: A Gentle Introduction. *Foundations and Trends in Machine Learning* (2023).

[3] Mustafa Arslan. 2026. Nexus: Depth-Adaptive KV-Cache Splicing and Retrieval-Decoupled Tool Routing for Agentic LLMs on Unified Memory. *arXiv preprint arXiv:2608.20397* (2026).

[4] Ethan Bito, Yongli Ren, and Estrid He. 2026. One Pass, Any Order: Position-Invariant Listwise Reranking for LLM-Based Recommendation. In *Proceedings of SIGIR*. arXiv:2604.27599.

[5] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting Language Models to Compress Contexts. In *EMNLP*.

[6] Chenlong Deng et al. 2024. A Silver Bullet or a Compromise for Full Attention? A Comprehensive Study of Gist Token-based Context Compression. *arXiv preprint arXiv:2412.17483* (2024). ACL 2025; TODO verify authors.

[7] Yichu Fang, Sitong Wei, Haozhe Hu, and Xiaoyu Shen. 2026. ReCache: Efficient KV Cache Reuse and Compression for Tool-Augmented LLM Agents. *arXiv preprint arXiv:2608.19662* (2026).

[8] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *ICLR*.

[9] Google DeepMind. 2026. Gemma 4 model card. https://huggingface.co/google/gemma-4-E2B. TODO verify technical report.

[10] hexgrad. 2025. Kokoro-82M text-to-speech. https://huggingface.co/hexgrad/Kokoro-82M, Apache-2.0.

[11] Junhao Hu et al. 2025. EPIC: Efficient Position-Independent Caching for Serving Large Language Models. In *ICML*.

[12] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *SIGIR*.

[13] Andrew Krikorian, Yayuan Li, and Jason J. Corso. 2026. NTILC: Neural Tool Invocation via Learned Compression. *arXiv preprint arXiv:2606.06566* (2026).

[14] Aditya Kusupati et al. 2022. Matryoshka Representation Learning. In *NeurIPS*.

[15] Zongqian Li, Yixuan Su, and Nigel Collier. 2024. 500xCompressor: Generalized Prompt Compression for Large Language Models. *arXiv preprint arXiv:2408.03094* (2024).

[16] Jeremiah Zhe Liu et al. 2020. Simple and Principled Uncertainty Estimation with Deterministic Deep Learning via Distance Awareness. In *NeurIPS*.

[17] Nelson F. Liu et al. 2024. Lost in the Middle: How Language Models Use Long Contexts. *TACL* (2024).

[18] Qi Liu, Bo Wang, Nan Wang, and Jiaxin Mao. 2025. Leveraging Passage Embeddings for Efficient Listwise Reranking with Large Language Models. In *Proceedings of the ACM Web Conference (WWW)*.

[19] Weitang Liu, Xiaoyun Wang, John D. Owens, and Yixuan Li. 2020. Energy-based Out-of-distribution Detection. In *NeurIPS*.

[20] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *NeurIPS*.

[21] Shishir G. Patil et al. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *ICML*. TODO verify.

[22] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. *NeurIPS* (2024).

[23] Yujia Qin et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *ICLR*.

[24] Qwen Team. 2025. Qwen3-Omni Technical Report. TODO verify arXiv ID.

[25] Revanth Gangi Reddy et al. 2024. FIRST: Faster Improved Listwise Reranking with Single Token Decoding. In *EMNLP*.

[26] Allen Z. Ren et al. 2023. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In *CoRL*.

[27] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. 2025. When2Call: When (not) to Call Tools. In *Proceedings of NAACL*. 3391–3409.

[28] RVtech. 2025. Audio2Tool: speech-to-tool-call evaluation set. Hugging Face dataset RVtech/Audio2Tool, CC BY-NC 4.0. TODO verify title, authors and paper.

[29] Mauricio Sadinle, Jing Lei, and Larry Wasserman. 2019. Least Ambiguous Set-Valued Classifiers with Bounded Error Levels. *J. Amer. Statist. Assoc.* (2019).

[30] Sarkar. 2026. ContextCache: Content-Hashed KV Caching of Tool Schemas. Zenodo record 18795189. TODO verify full title and author list.

[31] Ferdinand Schlatt et al. 2025. Set-Encoder: Permutation-Invariant Inter-Passage Attention for Listwise Passage Re-Ranking with Cross-Encoders. In *ECIR*. arXiv:2404.06912.

[32] Zhengliang Shi et al. 2025. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. In *Findings of ACL*. arXiv:2503.01763; TODO verify full author list.

[33] East Sun, Yan Wang, and Lan Tian. 2025. Block-Attention for Efficient Prefilling. In *ICLR*. arXiv:2409.15355; TODO verify title/authors.

[34] Weiwei Sun et al. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *EMNLP*.

[35] Xiangxi Tian and Ran Guan. 2026. Toollery: Scaling LLM Agents to Thousands of Skills and Tools. *arXiv preprint arXiv:2609.22218* (2026). EMNLP 2026.

[36] TODO verify authors. 2025. Long Context In-Context Compression by Getting to the Gist of Gisting. *arXiv preprint arXiv:2504.08934* (2025).

[37] TODO verify authors. 2026. The 99% Success Paradox: When Near-Perfect Retrieval Equals Random Selection. *arXiv preprint arXiv:2605.18857* (2026).

[38] TODO verify authors. 2026. Beyond Aggregate Risk: Role-Stratified Conformal Risk Control for LLM Tool Calls. *arXiv preprint arXiv:2607.24343* (2026).

[39] TODO verify authors. 2026. Ranked by Position: Order Sensitivity as an Exploitable Attack Surface in LLM Listwise Recommenders. *arXiv preprint arXiv:2607.24869* (2026).

[40] TODO verify authors and exact title. 2025. Tool-DE: document expansion for tool retrieval, with Tool-Embed and Tool-Rank. *arXiv preprint arXiv:2510.22670* (2025).

[41] Feng Wang, Yuqing Li, and Han Xiao. 2025. jina-reranker-v3: Last but Not Late Interaction for Listwise Document Reranking. *arXiv preprint arXiv:2509.25085* (2025).

[42] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2025. ToolGen: Unified Tool Retrieval and Calling via Generation. In *ICLR*. arXiv:2410.03439.

[43] Zhiyong Wu et al. 2024. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. *arXiv preprint arXiv:2410.23218* (2024). TODO verify.

[44] Jingbo Yang et al. 2025. KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse. *arXiv preprint arXiv:2502.16002* (2025).

[45] Xinyu Yang, Tianqi Chen, and Beidi Chen. 2025. APE: Faster and Longer Context-Augmented Generation via Adaptive Parallel Encoding. In *ICLR*.

[46] Jiayi Yao et al. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *EuroSys*.

[47] Donghui Zha, Lingwei Xu, Linxiao Wu, Yixue Dong, and Haochen Li. 2026. CacheRouter: A Dual-Path Tool Routing Architecture with Cache-Preserving Main-Model Isolation for Long-Tail Tool Discovery. *arXiv preprint arXiv:2608.22708* (2026).

[48] Peitian Zhang et al. 2024. Long Context Compression with Activation Beacon. *arXiv preprint arXiv:2401.03462* (2024).

[49] Yanzhao Zhang et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).

[50] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. 2024. Large Language Models Are Not Robust Multiple Choice Selectors. In *ICLR*.